

Fathom: Per-Query Read Depth for Sparse Decoding over Offloaded KV Caches

Vivek Kalyanarangan

Abstract

When agentic sessions run to a million tokens with many sessions resident at once, the KV cache and the index that ranks it live in host memory, and the scan that ranks all n keys for a top- k step becomes the traffic that bounds decoding. We present Fathom, a key scan in which each query decides how many bits of each key channel to read. The 4-bit K cache is stored channel-major as bit planes, so a prefix of t planes is exactly the channel’s t -bit quantizer, and the query spends its bit budget by reverse water-filling over the variance-weighted importance of its channels. At one million tokens on Qwen3-8B a decode step is $1.67\times$ faster in GPU time than with the 136-bit scans of Double Sparsity, Loki and SparQ $r = 32$, and in the same GPU time as SparQ’s 68-bit read ($r = 16$) Fathom reads 18% fewer bytes with lower attention error on six of seven model and context settings. On RULER-style tasks every per-token scan matches exact top- k decoding, and on real coding-agent sessions Fathom reaches the step agreement of the most accurate 136-bit scan at 92 bits. The store is the 4-bit K copy a quantized serving stack already holds, and the method is not faster when the index is resident in GPU memory.

1 Introduction

A coding or browsing agent carries a context of hundreds of thousands of tokens for hours, most of it cached rather than newly generated, and a server hosts many such sessions at once. Their KV caches no longer fit beside the weights in GPU memory and are held in host memory or a slower tier, so a decode step is bounded by what crosses the interconnect. Decoding was already bandwidth-bound, since each generated token reads the model weights once and, at long context, the entire KV cache once per layer. Offloading makes the cache read the dominant term. Grouped-query attention (GQA) [1] and 2- to 4-bit KV quantization [11, 6] shrink the cache. Top- k sparse attention [16, 22, 15, 18] shrinks the *read*, fetching only the k keys and values with the highest attention scores. Since the scores are unknown before the read, every such method first scans a cheap representation of all n keys to rank them. The scan is a stream of n small records whose cost is measured in bits per token, and it grows with n while the fetch of the winners does not.

The numbers for Qwen3-8B (36 layers, 8 KV heads of 128 channels, 4 query heads per KV head) at 32k tokens illustrate the split. Dense attention reads the whole bf16 KV cache, 4.8GB per step. Top- k with $k = 512$ per query head fetches the union of the four heads’ winners, at most 2048 rows per KV head and about 200MB per step in our runs, a cost that does not grow with the context. A Loki, Double Sparsity or SparQ ($r = 32$) scan reads 32 coordinates of each key at 4 bits plus a

block scale, 136 bits per token for each of the 288 layer-head pairs, 557KB per pair and 160MB per step at 32k. That term grows linearly with context, to 5.1GB at one million tokens, while the winner rows stay near 200MB. All bit counts in this paper are per token per KV head per layer, under the one convention of §4.

Two levers reduce the scan. One scores fewer things, such as pages, blocks or landmarks [18, 20, 17]. The other reads fewer bits per key. Existing per-token scans fix the bits in advance. Loki [16] reads r principal-component coordinates of every key; Double Sparsity [22] reads c channels chosen offline from a 4-bit label cache; SparQ [15] lets the query pick the r channels with the largest summed $|q|$ over the query heads of the group and reads them at full depth; thumbnail scans [10] read every channel at 2 bits. In all of them the depth of the read is the same for every channel the method touches.

This paper lets the read depth be decided per query and per channel. Two observations make this possible. First, quantization error falls by $4\times$ per bit. If channel j of a key has been read to a depth of t_j bits, one more bit reduces the remaining score error by an amount proportional to $g_j 4^{-t_j}$, where g_j is that channel’s contribution to the score variance for this query. The first bit of an important channel is worth far more than its fourth, and the fourth bit of an important channel can be worth less than the first bit of a minor one. Allocating bits in order of this marginal value is reverse water-filling [5], and it has a closed form. Second, if the 4-bit K cache is stored as bit planes, channel-major, the first t planes of

a channel *are* its t -bit mid-rise quantizer with the same block scale, so a prefix read is exact and contiguous and no lower-precision copy is needed. Together they turn a 4-bit copy of the keys into a multi-resolution index that each query reads to the depth it needs.

We evaluate Fathom against Loki, Double Sparsity, SparQ, a 2-bit thumbnail and a block-landmark index on 7 model and context settings, on RULER-style tasks at 32k and 128k, and in the hardware regime the method is built for as well as the ones it is not. The byte saving holds on every setting; it becomes a time saving when the scan bytes cross PCIe, and §7 analyses when that is.

Contributions.

- A measured result in the regime that motivates the work: with the KV cache and its index in host memory, a decode step is $1.37\times$ faster at 256k tokens and $1.67\times$ at 1M in GPU time than with the 136-bit scans, at equal or better accuracy than Double Sparsity and Loki; against SparQ’s 68-bit read it reads 18% fewer bytes in the same GPU time and is $1.1\text{--}5.3\times$ more accurate (§5.1, §5.2).
- A bit-plane key store in which prefix reads are exact lower-precision quantizers, and a per-query reverse water-filling rule for the read depth of every channel, with an optional per-layer budget calibrated once (§3).
- Equal-error byte savings of $1.8\text{--}2.9\times$ against Double Sparsity’s 136-bit scan on 7 settings up to 128k tokens, downstream parity with the exact top- k oracle on RULER-style tasks, and on real coding-agent sessions the step agreement of the most accurate 136-bit scan at 92 bits (§5.3, §5.5, §5.6).
- A basis rule that uses raw channels for models with QK-norm (query and key normalisation, as in Qwen3) and planes of the Karhunen–Loève transform (KLT) of the keys otherwise (Llama-3.1, Qwen2.5) (§3.5).
- An analysis of the HBM-resident case: with the index in GPU high-bandwidth memory (HBM) the method is not faster, and an arithmetic-per-byte analysis says why (§7), with ablations of every design choice (§6, Appendix D).

Summary of measured results. Fathom is built for one situation: sparse decoding when the KV cache and the scan index are in host memory. Table 1 summarises the measurements in that regime and in the regimes where the method does not help.

2 Background and Related Work

Top- k decoding. Given a query $q \in \mathbb{R}^D$ and keys $K \in \mathbb{R}^{n \times D}$, exact attention weights are $\text{softmax}(qK^\top / \sqrt{D})$.

Attention mass concentrates on few keys, so H₂O [24] and StreamingLLM [21] evict, while Loki, Double Sparsity, SparQ and Quest keep the full cache and select at decode time by approximate score. We evaluate every method under one protocol: the first 4 tokens and the last 32 are always kept, following StreamingLLM and H₂O, and the remaining $k - 36$ are the top-scoring keys under the method’s scores, per query head; each method is measured by the exact attention output over its selected set. SparQ’s reallocation of the unselected attention mass onto a mean value is not applied to any method.

Loki. Loki [16] rotates keys into the principal-component (PCA) basis of calibration keys and scores queries against the first r coordinates, kept in the model’s precision; the original method stores no second copy of the keys and its $r = 32$ read is 512 bits per token. To compare byte for byte with 4-bit scans we quantize the r coordinates to 4 bits with the same block scales as every other method (136 bits at $r = 32$), and we order the basis by calibration score variance rather than by eigenvalue, which favours Loki. Its accuracy is rank-limited: on Qwen3-8B the 32-coordinate basis misses late-layer directions at 32k and fp16 coordinates do not repair it (§6).

Double Sparsity. Double Sparsity [22] keeps a label cache of c channels per head chosen offline from a calibration statistic of the score contribution and scans it at 4 bits. We choose channels by the per-head product of mean $|q|$ and mean $|k|$ on calibration text and run $c = 32$, a quarter of the channels; the paper’s own default is a sixteenth, eight channels, and it also reports a half and all of them. At $c = 32$ the label cache reads 136 bits per token with fp16 block scales, the same as the other fixed-depth scans, and is 17 bytes per token stored beside the K cache. It is the strongest offline baseline on the Qwen3 models; Loki is stronger on the other three models.

SparQ. SparQ [15] lets each query choose the r channels with the largest $|q_j|$ and reads them from a channel-major K store. Under grouped-query attention its published rule sums $|q|$ over the query heads that share a KV head before the top- r , so one set of r channels is read per KV head. SparQ’s own store is fp16 (512 bits at $r = 32$); with the 4-bit codes we give every scan (§4) that is 68 bits at $r = 16$ and 136 at $r = 32$. We implement that rule. A stronger variant that lets every query head pick its own channels and reads the union of the picks is not SparQ’s method; we report it once as an ablation (§6). SparQ also sums the approximate scores over the group before its top- k ; we do not adopt that for any method and select top- k per query head throughout.

Table 1: Measured results by regime on an A100 (Qwen3-8B, $k = 512$). GPU time per decode step; the 136-bit scans are Double Sparsity, Loki and SparQ $r = 32$.

regime	what binds	Fathom (56-bit read unless noted)
KV rows and index in host memory, 1M tokens	PCIe bytes	1.67 $\times$ faster than the 136-bit scans, 2.50 $\times$ than landmarks; the same GPU time as SparQ $r = 16$ at 56 bits (ratio 1.05) and 1.11 $\times$ faster at 47 bits, at lower error
same, 128k tokens, real prefill	PCIe bytes	1.26 $\times$ faster than the 136-bit scans; SparQ $r = 16$ takes 1.00 $\times$ its GPU time
real coding-agent sessions, 100k tokens, $k = 2048$	scan fidelity	step agreement with exact top- k 0.67 , SparQ $r = 16$ 0.49, landmarks 0.47
real coding-agent sessions, 100k tokens, $k = 512$	scan fidelity	0.60 at 92 bits, equal to SparQ $r = 32$'s 0.60 at 136 bits
rows in host memory, index in HBM	the shared row fetch	not faster; all per-token scans land at 45–48 ms
everything in HBM	arithmetic per scanned bit	not faster; the scan kernel is 1.4 $\times$ slower than a nibble scan

Block and landmark selection. Quest [18] scores 16-token pages by per-channel minimum and maximum keys; ShadowKV [17] scores 8-token chunks by their mean key, keeps a low-rank K cache on the GPU and offloads V; InfLLM [20] scores blocks by representative tokens. This is the “score fewer things” lever, orthogonal to ours. Our landmark baseline follows ShadowKV, the mean key of each 8-token block in fp16, 256 bits per token, run at the paper’s k rather than at the block budgets those systems use. Both systems keep their landmark index in GPU memory; the host-memory setting of §5.1 is ours, not theirs.

Thumbnail scans. FFD [10] splits K into a 2-bit thumbnail and an 8-bit residual, selects by a score threshold and fuses scan and attention in one kernel. Our 2-bit thumbnail baseline reads all 128 channels at two planes (288 bits per token) and selects top- k with exact attention on the selected set, which is more generous than FFD’s own scoring; the bit-plane store makes it a special case of ours with uniform depth.

KV quantization. KIVI [11] (2-bit, per-channel keys and per-token values), KVQuant [6] (non-uniform, pre-RoPE, dense-and-sparse) and TurboQuant [23] (random rotation, Lloyd–Max scalar quantizer and a 1-bit residual) compress the cache itself; Any-Precision LLM [13] stores weights in bit planes so that a prefix is a lower-precision model. Our layout applies the bit-plane idea to the K cache and reads a per-query prefix per channel.

Offloaded retrieval. MagicPIG [4] samples with locality-sensitive hashing (LSH) tables on the CPU, RetroInfer [3] and RetrievalAttention [9] keep vector indexes in host memory, and InfiniGen [8] prefetches speculatively from a host-resident cache. This is the regime where scan bytes are time and where our gains are measured.

3 Method

3.1 Cost model

Let N be the number of sequences decoding together, each with a context of n tokens (so Nn tokens are resident), b the scan bits per token per KV head, H the KV heads, L the layers, G the query heads per KV head, $k_{\cup} \leq Gk$ the number of distinct winner rows per KV head after the union over the group’s picks, and R the bytes of a full K+V row. A decode step reads

$$\text{bytes} \approx W + N L H \left(\frac{b}{8} n + k_{\cup} R \right), \quad (1)$$

where W is the weight read once per step. The scan term grows with the resident tokens Nn ; the row term grows with N only, since each sequence fetches at most $k_{\cup}$ rows per head whatever its length. Reducing b matters exactly when $b n / 8$ is the largest of the three terms, which is the long-context, many-session regime this paper targets.

3.2 Bit-plane K cache

Keys are quantized once to 4 bits with a uniform, symmetric code per channel and one fp16 scale per 64-token block. Uniformity is what makes a prefix of bit planes an exact coarser quantizer; a non-uniform code would not have this property. For block β and channel j let $a_{\beta,j} = \max_{i \in \beta} |k_{i,j}|$ be the block maximum and $s_{\beta,j} = a_{\beta,j} / 8$ the cell width; the code is

$$c_{i,j} = \text{clip}(\lfloor k_{i,j} / s_{\beta,j} \rfloor, -8, 7) + 8 \in [0, 16), \quad (2)$$

so that code 8 is zero and the most significant bit is the sign.

Bit p ($0 = \text{most significant}$) of the 64 codes of block β , channel j , is packed into one 64-bit word $P_{\beta,j,p}$, the plane. Planes are stored channel-major, so the words $P_{\cdot,j,0 \dots t-1}$ are contiguous over the sequence. Reading the first t planes of a channel gives $c^{(t)} = c \gg (4-t)$, and the dequantized value $(c^{(t)} + \frac{1}{2} - 2^{t-1}) a_{\beta,j} / 2^{t-1}$ is exactly

the t -bit mid-rise quantizer of the range $[-a_{\beta,j}, a_{\beta,j}]$ with cells of width $a_{\beta,j}/2^{t-1}$; at $t = 4$ it reduces to $(c - 8 + \frac{1}{2}) s_{\beta,j}$. The store is a 4-bit copy of K with fp16 block scales, 68 bytes per token per KV head. In a serving stack that keeps its K cache in 4-bit form the planes can serve as that cache; our experiments keep bf16 rows for the winners and treat the planes as a separate index, and the memory comparison in §7 charges the full 68 bytes. Figure 1 shows the layout.

Compatibility with other KV quantizers. The layout requires a scalar code per channel whose bit prefixes are coarser quantizers; it is not agnostic to the quantization family. A fixed rotation before quantization is compatible, and §3.5 uses one, but the rotation must concentrate variance (a KLT) rather than spread it (the random rotation of TurboQuant [23]), because water-filling has nothing to allocate when every coordinate carries equal variance; TurboQuant’s Lloyd–Max quantizer is in addition non-uniform. Uniform integer KV formats, including KIVI’s per-channel codes with a zero point [11] and INT8, are compatible directly, since a prefix of a uniform code is a coarser uniform code. Non-uniform codes lose the exactness property. KVQuant’s lookup-table datatypes [6] would need their table stored in value order and a per-depth table, and its keys quantized before the rotary position embedding (RoPE) and its separate outlier component would each need handling we have not built; FP8, whose prefixes are sign and exponent bits, is monotone in magnitude and would likewise need measured rather than 4^{-t} marginal gains. Codebook vector quantization has no per-channel bit depth; residual VQ, being progressive by stage, would admit a per-query depth in stages, which we do not explore.

3.3 Per-query read depth

For the group $\{q^{(h)}\}_h$ of query heads sharing one KV head, the contribution of channel j to the scores has variance across keys proportional to

$$g_j = \sum_h (q_j^{(h)})^2 \text{Var}(k_j), \quad (3)$$

with $\text{Var}(k_j)$ measured once on calibration keys. Reading t planes of a channel is a t -bit uniform quantizer over $[-a_{\beta,j}, a_{\beta,j}]$: it splits that range into 2^t cells of width $\Delta = 2a_{\beta,j}/2^t$, and a value is off from its cell centre by an error of variance $\Delta^2/12 = a_{\beta,j}^2/(3 \cdot 4^t)$. Every extra plane therefore divides the error variance of that channel by four. That error enters the score multiplied by q_j , so summed over the heads of the group, and with $a_{\beta,j}^2$ standing in for $\text{Var}(k_j)$, channel j contributes an expected squared score error proportional to $g_j 4^{-t_j}$, and the total is $\sum_j g_j 4^{-t_j}$. Minimizing it under $\sum_j t_j \leq B$ is reverse water-filling [5]: bits go first to the channels

with the largest g_j , and each channel’s depth is set by how far its importance sits above a common water line θ . With integer depths the optimum is

$$t_j = \text{clip}(\text{round}(\log_4(g_j/\theta)), 0, 4), \quad \sum_j t_j \leq B, \quad (4)$$

Each t_j is a step function of θ that falls as θ rises, so $\sum_j t_j$ is too; the water line is the smallest θ at which the sum fits the budget, and 30 bisection steps on $\log \theta$ per query group locate it. Channels with $t_j = 0$ are skipped entirely and the rest are the active channels; a read at mean 48 bits touches 30–34 of 128 channels at 1–4 planes each on the models we test (Table 21). The plan is shared by the G heads of the group, so the K bytes are read once for all of them. Two choices differ from SparQ: g_j weights the query by the key variance rather than ranking by $|q_j|$, and depth is graded rather than all-or-nothing. Appendix A works Eq. 4 through a six-key example.

3.4 Per-layer budgets

Layers differ in how peaked their score distributions are. The flat budget gives every layer the same B . A greedy allocation on calibration text instead assigns budgets $B_\ell \in \{24, \dots, 128\}$ to layers at a target mean (48 or 64 bits), each step moving bits to the layer with the largest error drop per bit; we call this the per-layer plan. On the Qwen3 models it lowers error at mean 48 by about half at 16k and by 8–11% at 32k, and is worse at mean 64 on Qwen3-8B at 32k with $k = 128$; it is worse on Qwen2.5-7B and Llama-3.1-8B (up to $1.7\times$ and $2.1\times$ at mean 64) and neutral to better on Qwen2.5-7B-1M, and a plan calibrated at 16k evaluated at 32k is worse than the flat budget (Tables 14 and 10, §6). Our recommendation is a flat budget by default: it needs no calibration and has no context-length dependence. The per-layer plan is a tuning step for a fixed deployment, calibrated at that deployment’s context length; we report both throughout.

3.5 Basis rule

Water-filling over raw channels assumes score variance is concentrated in few channels. On models with QK-norm (Qwen3) it is, and rotating keys with the calibration KLT spreads the per-query sparsity and raises error. On models without QK-norm (Llama-3.1-8B, Qwen2.5-7B, Qwen2.5-7B-1M) the same rotation lowers error at equal bits (Table 11). The rotated variant stores planes of $(k - \mu)V$ with the score-variance-ordered eigenbasis V of the calibration keys, per KV head and layer, and uses $g_j = \sum_h ((q^{(h)}V)_j)^2 \lambda_j$; nothing else changes. Unlike Loki nothing is truncated; all D rotated coordinates are stored, and the query decides how deep to read each. The rule is decided once per model by evaluating both stores on calibration text, and every result below applies

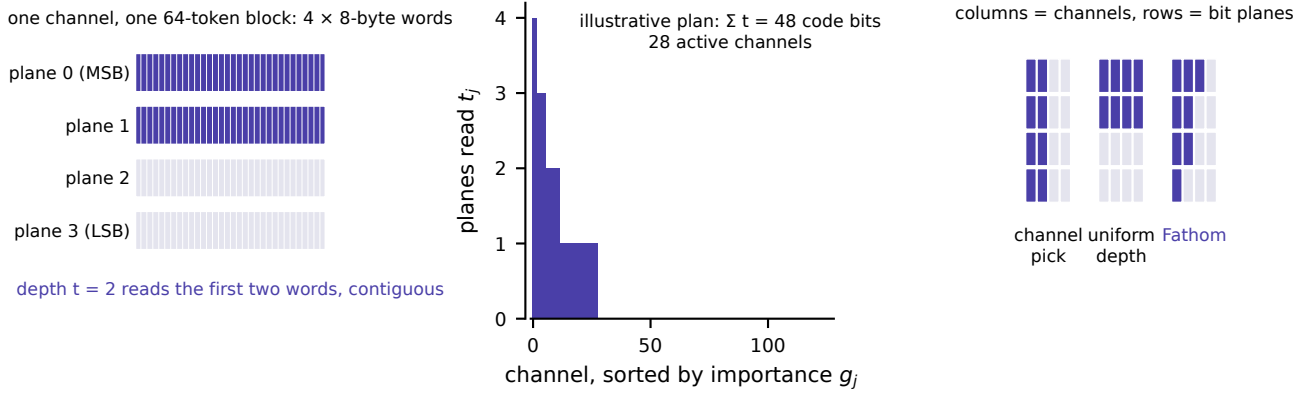


Figure 1: Left: one channel of one 64-token block as four bit planes; a depth- t read is the first t words. Middle: an illustrative per-query read of a 128-channel key at a mean of 48 bits, a staircase over channels sorted by importance g_j (the measured number of active channels is 30–34, Table 21). Right: the read shapes of channel picks (Double Sparsity, SparQ), uniform depth (thumbnails) and ours.

it: raw planes on Qwen3, rotated planes on the other three models.

3.6 Reading a host-resident store

When the store lives in host memory, a GPU kernel reading mapped host memory word by word is limited by the small, scattered transactions, whereas a contiguous copy runs at the link rate (Fig. 10). With the whole sequence as one channel-major block, the first t_j planes of channel j are one contiguous run of $t_j n/8$ bytes. A gather kernel copies one run per active channel, plus that channel’s scales, into a staging buffer, and the scan runs in HBM; the run list has a fixed size, one slot per channel with inactive slots of length zero, so no host synchronization is needed. The same transfer is given to every baseline in the offload experiments.

4 Experimental Setup

Models and hardware. Qwen3-8B (16k and 32k), Qwen3-4B (16k) [14], Qwen2.5-7B (32k) and Qwen2.5-7B-Instruct-1M (32k and 128k), with activations captured, fidelity computed, RULER-style tasks run and all timing measured on one NVIDIA A100-SXM4-80GB pod (PCIe 4.0, 2 TB host RAM). Llama-3.1-8B [12] at 4k uses activations captured on an NVIDIA L4 and evaluated with the same fidelity code. Calibration uses the Wikitext-103 train split and evaluation the test split, both at the evaluated context length.

Fidelity metric. For 128 decode positions at the end of the window (64 at 128k), all layers and KV heads, the selected set is sink $4 + \text{local } 32 + \text{top-}(k - 36)$ by the method’s scores, per query head; we report the mean relative L_2 error of the attention output from that set against dense attention. k scales with context (128 at

4k, 256 at 16k, 512 at 32k) unless stated; §5.4 varies it. Each setting is one held-out Wikitext window.

RULER-style tasks. These synthetic retrieval and state-tracking tasks are our proxy for the long-range recall that long agent sessions depend on; they share structure with coding and tool-use transcripts, not content, and §8 says what a workload-level evaluation would need. Tasks with RULER templates [7] on a Wikitext haystack: single-needle, multi-key, multi-value and multi-query needle-in-a-haystack, variable tracking and frequent-word extraction at 32k (Qwen3-8B, $k = 128$, 40 samples per task), and multi-key, multi-query, variable tracking and frequent-word extraction at 128k (Qwen2.5-7B-Instruct-1M, $k = 128$, 20 samples). One dense prefill per sample, then greedy decoding branched per method from the same KV cache with sparse attention on every generated token. Score is the fraction of gold strings present in the generation. The decode scorers reproduce the fidelity code to 10^{-4} and the dense path reproduces HuggingFace generation token for token, checked on this pod before the runs.

Offload harness. Model weights on the GPU; K/V rows and, unless noted, the scan store in pinned host memory. Each step runs the scan, the top- k per query head, the gather of the union of the selected rows over PCIe, and exact attention. The primary timing metric is GPU time, the profiler’s kernel plus memcpy time of a decode step, because it measures the work the method changes. Wall-clock is the median of the steps after two warm-up steps. It adds the host-side time of this research harness, which differs by method and would not exist in a fused implementation. Table 2 reports both and Table 3 GPU time. The synthetic runs of Table 2 and Figure 2 use a synthetic KV cache (calibration keys tiled to n , random values), because no model here prefills

a million tokens, and are timing only; the 32k–128k runs of Table 3 use a real prefill.

Baselines. Double Sparsity $c = 32$, Loki $r = 32$ and $r = 64$, SparQ $r = 16$ and $r = 32$ under its grouped-query rule, a 2-bit all-channel thumbnail, the ShadowKV-style block-mean landmark index with 8-token blocks, the exact top- k oracle and dense attention. All scans use 4-bit codes with identical block scales. In the timing harness the 16- and 32-channel scans select their channels per query by the group’s $\sum |q|$, so the 32-channel row stands for SparQ $r = 32$ and for the Loki and Double Sparsity byte count at once.

Byte accounting. A single convention applies throughout. A method’s bits per token are the code bits it reads plus 16 bits per 64 tokens for the fp16 block scale of every channel it reads. That gives 136 for 32 channels at 4 bits, 68 for 16, 288 for the 2-bit thumbnail, 544 for the full 4-bit scan and 256 for the fp16 landmark; for ours it is the planes read plus the scales of the active channels, which the fidelity tables report to the nearest bit and which is exactly what the timing harness transfers.

5 Results

§5.1 measures the target regime, a decode step with the KV cache and the scan index in host memory; §5.2 compares with SparQ $r = 16$, the scan that takes the same GPU time; §5.3 to §5.5 establish the accuracy side at equal bytes, across selection ratios and downstream.

5.1 The target regime: decoding with the KV cache and its index in host memory

With the index in host memory the scan share of the step grows with context and the byte ratio approaches the time ratio (Table 2, Fig. 2). At 1M tokens the step’s GPU time with our 56-bit read is $1.67\times$ lower than with the 32-channel scan, which is the SparQ $r = 32$, Double Sparsity and Loki byte count, $2.50\times$ lower than with the landmark index and $3.12\times$ lower than with the thumbnail; in wall-clock the ratios are 1.38, 2.07 and $2.59\times$. SparQ $r = 16$ moves 22% more scan bytes than our 56-bit read and takes $1.05\times$ its GPU time: our scan kernel does more arithmetic per byte than a nibble scan, and at this link rate that offsets most of the transfer saving; its error is higher on every setting (§5.3). Our mean-40 read, at 31% fewer bytes than SparQ $r = 16$, is $1.11\times$ faster. In wall-clock SparQ $r = 16$ is faster: the gap between wall-clock and GPU time is 48ms per step for our 56-bit read against 24ms for SparQ (Table 2; A6 in §6 discusses this gap). Our mean-64 read (74 bits)

costs $1.21\times$ the 56-bit read. At 256k the shared row fetch, 167–216MB per step across methods, is a large part of the step and GPU time favours the 56-bit read over the 32-channel scan by $1.37\times$. With a real prefill at 32k–128k the ordering is the same, $1.26\times$ over the 32-channel scan at 128k in GPU time and $1.00\times$ against SparQ $r = 16$ (Table 3, left). At batch 2 with the synthetic cache the GPU-time ratio over the 32-channel scan is $1.44\times$ at 256k and $1.63\times$ at 512k.

Table 4 splits the 1M step into components. Every scan moves its bytes at the link rate, about 26 GB/s, so the transfer column is the byte count over the link rate and is where Fathom saves; its scan kernel costs about twice SparQ’s per byte because it extracts bits, and the shared costs of top- k selection, the winner-row fetch and the weight GEMMs are 33% of Fathom’s step and 20% of the 32-channel scan’s. Against SparQ $r = 16$ the 56-bit read’s transfer saving is mostly offset by its extra kernel time, which is the small margin in the table; the mean-40 read, at 31% fewer scan bytes than SparQ $r = 16$, takes $1/1.11$ of its GPU time and $1/1.77$ of the 32-channel scan’s.

Control: index in HBM. If the same indices are kept in HBM and only the rows are offloaded (Table 3, right; Fig. 8), the per-token scans land at 45–48ms per step at 128k, the landmark index at 44ms and the thumbnail at 55ms, and ours is not faster. The time advantage therefore requires the index itself to live in the slower tier, which is the case of long contexts with many concurrent sessions, where a 17-byte-per-token label cache (5.1 GB per million-token sequence for Qwen3-8B) or a 32-byte-per-token landmark index (9.7 GB) does not fit next to the weights.

5.2 Comparison at matched GPU time

Among the scans in Table 2, SparQ at $r = 16$ is the one whose step time matches ours: it reads 68 bits per token to our 56, and the two take the same GPU time per step, SparQ $1.05\times$ ours at 1M tokens (Table 2) and $1.00\times$ at 128k with a real prefill (Table 3). This section therefore holds time fixed and compares bytes and accuracy. At equal time Fathom reads 18% fewer scan bytes and has 1.1 – $5.3\times$ lower attention-output error on all seven settings with the better of its two plans, and on six with the flat default (Table 5). The margin is largest at the selection ratio that matters downstream ($5.3\times$ at $k = 2048$ on Qwen2.5-7B-1M at 128k) and smallest on Qwen3-8B at 32k with the flat budget, where the two are close to a tie. At the mean-40 budget, about 47 bits, Fathom’s error is lower on 6 of the 7 settings and its step is faster, SparQ $r = 16$ taking $1.11\times$ its GPU time at 1M.

Two measurements qualify this. In wall-clock, SparQ $r = 16$ is 7% faster at 1M; the difference between wall-

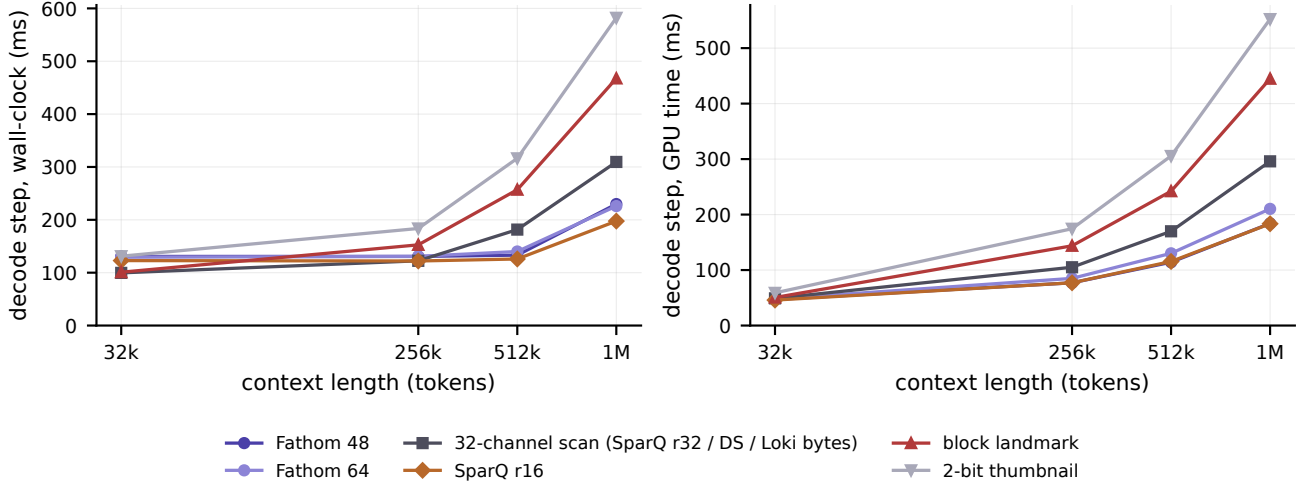


Figure 2: Decode step versus context with K/V rows and scan index in pinned host memory (A100, Qwen3-8B, $k = 512$, batch 1, synthetic KV at every point). (a) wall-clock, (b) GPU time. Every method’s index is moved with the contiguous gather of §3.6.

Table 2: Decode step (ms) on an A100 with K/V rows and the scan index in pinned host memory (Qwen3-8B, $k = 512$, batch 1, synthetic KV). bits is the measured scan traffic per token; GB/step is the measured PCIe traffic at 1M. GPU time is the profiled step; wall-clock is the median of the timed steps; ratios are computed from unrounded times.

method	bits	256k		512k		1M		1M relative to Fathom 48		
		wall	GPU	wall	GPU	wall	GPU	GB/step	wall	GPU
Fathom 40	47	129	73	129	106	224	168	1.96	1.00	0.95
Fathom 48	56	129	77	130	113	225	177	2.32	1.00	1.00
Fathom 64	74	129	85	140	130	230	215	3.00	1.02	1.21
SparQ $r=16$	68	121	78	126	115	210	186	2.76	0.94	1.05
32-channel scan	136	122	105	180	170	310	296	5.33	1.38	1.67
block landmark	256	153	144	257	242	466	444	9.89	2.07	2.50
2-bit thumbnail	288	184	175	316	306	583	554	11.09	2.59	3.12

clock and GPU time is host-side overhead of this research harness (48 versus 24 ms per step, §6 A6), not transfer or kernel time. With every index resident in HBM the two methods take the same time (§7). SparQ at $r = 32$, 136 bits, is more accurate than our 56-bit read on three of the seven settings (Qwen3-8B at 32k, Qwen2.5-7B and Qwen2.5-7B-1M at 32k) and costs $1.67\times$ its GPU time; the 74- to 92-bit reads reach its accuracy there, the 74-bit read at a $1.4\times$ saving (Table 2).

5.3 Fidelity at equal error

Table 6 and Figure 3 give the bits at which our scan reaches the error of Double Sparsity and of SparQ $r = 32$, both at 136 bits. Double Sparsity’s error is reached at 46–74 bits on all 7 settings, a $1.8\text{--}2.9\times$ saving; SparQ $r = 32$, the strongest fixed-depth scan on six of the seven settings, is reached at 38–92 bits. SparQ $r = 16$ at 68 bits has higher error than our 56-bit read on every

setting, by the flat budget on 6 of them and by the per-layer plan on the rest (Tables 14 and 15). Loki at 136 bits is matched at 56–75 bits on the 4 settings where it is not rank-limited; on the Qwen3 models it collapses (§6). Tables 14 and 15 list every method’s row, with the full 4-bit scan as the floor.

5.4 Effect of the selection ratio

A 128k run at $k = 128$ selects 0.1% of the keys; the fixed-depth scans’ error rises $5\text{--}6\times$ relative to the 1.6% ratio, ours $15\times$ and the full 4-bit scan’s far more, so the margin over SparQ $r = 16$ narrows from $4.4\times$ at 1.6% to $1.6\times$ at 0.1%; at the matched ratio ($k = 2048$) the crossings with the 136-bit scans are where they are at 32k (Table 16, Fig. 6). The saving does not shrink with context length; it shrinks with the selection ratio, for every scan, and at 0.1% the rotated store lowers the 56-bit error from 0.0300 to 0.0196.

Table 3: Real prefill, Qwen3-8B, $k = 512$, batch 1, A100: GPU time per step (ms) with the scan index in host memory and with the same index kept in HBM (rows in host memory in both). The 32-channel scan is the SparQ $r = 32$, Double Sparsity and Loki byte count.

method	index in host memory			index in HBM		
	32k	64k	128k	32k	64k	128k
Fathom 40	47	49	56	43	44	47
Fathom 48	47	50	58	43	44	47
Fathom 64	48	52	62	44	45	48
SparQ $r=16$	46	50	58	42	43	45
32-channel scan	50	57	73	43	44	47
block landmark	51	64	89	39	42	44
2-bit thumbnail	60	76	108	46	48	55
dense (all rows)	216	411	804	215	413	807

Table 4: Where the 1M-token step goes: GPU time per component (ms) from the profiler trace of the host-index run, Qwen3-8B, batch 1. “scan total” is transfer plus kernel, the part a scan method changes; top- k , row fetch and the weight matrix multiplications (GEMMs) are shared by every method. The landmark index scores with a GEMM, counted in the GEMM column.

method	scan transfer	scan kernel	top- k	row fetch	GEMMs	other	scan total	step
Fathom 40	68	19	36	15	10	19	88	168
Fathom 48	78	21	35	15	10	19	99	177
Fathom 64	107	27	36	16	10	19	134	215
SparQ $r=16$	96	11	36	15	10	19	106	186
32-channel scan	197	20	36	15	10	19	216	296
block landmark	359	0	7	9	29	40	359	444
2-bit thumbnail	406	67	35	17	10	19	473	554

Table 5: Fathom’s 47-bit and 56-bit reads and SparQ’s 68-bit read ($r = 16$), the two scans that take the same GPU time per step: attention-output error on the seven headline settings. Bold marks the lower error; the ratio uses the better of the flat budget and the per-layer plan.

Model	ctx/ k	SparQ $r=16$	Fathom, mean 40 (≈ 47 bits)			Fathom, mean 48 (≈ 56 bits)		
		68 bits	flat	per-layer	ratio	flat	per-layer	ratio
Qwen3-8B	16k/256	0.0077	0.0075	0.0033	2.3×	0.0052	0.0021	3.7×
Qwen3-8B	32k/512	0.0080	0.0142	0.0091	0.9×	0.0081	0.0074	1.1×
Qwen3-4B	16k/256	0.0071	0.0050	0.0027	2.7×	0.0034	0.0019	3.8×
Llama-3.1-8B	4k/128	0.0044	0.0013	0.0012	3.8×	0.0009	0.0010	4.9×
Qwen2.5-7B	32k/512	0.0090	0.0065	0.0096	1.4×	0.0044	0.0063	2.0×
Qwen2.5-7B-1M	32k/512	0.0091	0.0052	0.0075	1.7×	0.0038	0.0038	2.4×
Qwen2.5-7B-1M	128k/2048	0.0055	0.0019	0.0017	3.2×	0.0013	0.0011	5.3×

5.5 Downstream quality

With 40 samples per task at 32k and 20 at 128k, the standard error of a method’s mean score is about 0.014 and 0.034. Every per-token scan lies within 0.008 of the exact top- k oracle at 32k and within 0.025 at 128k, inside that error, and ours does so at 56 and 74 bits (Table 7, Fig. 7). The block-landmark index loses 0.043 against the oracle at 32k and 0.130 at 128k, because when k is

small relative to the block a block’s mean hides its best token. No per-token scan separates from another at this k ; differences between them appear only in attention error and in bytes. These tasks validate the scan as a retrieval mechanism over long contexts; task success on real coding or tool-use sessions is not measured; §5.6 measures step agreement on such sessions.

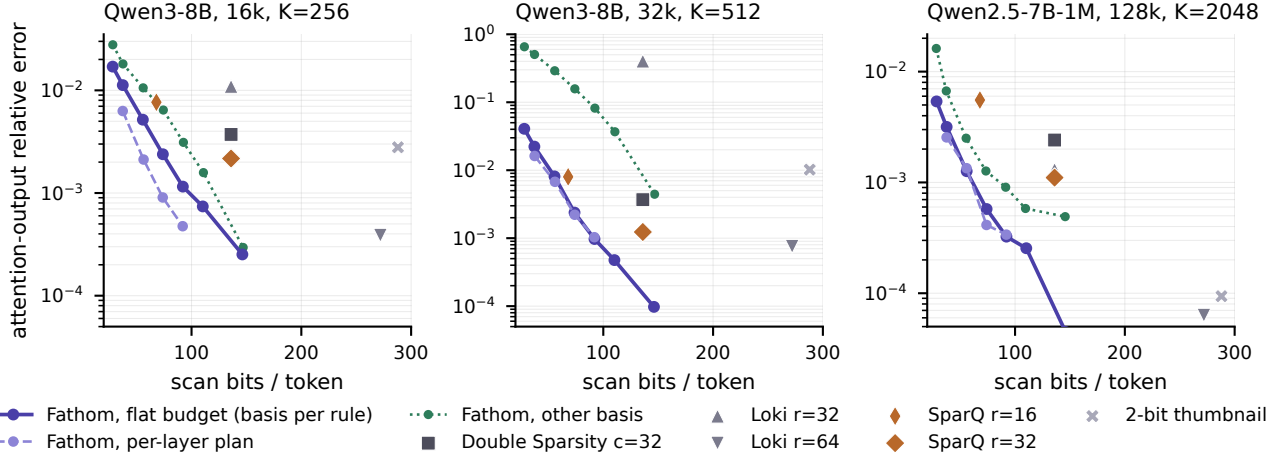


Figure 3: Attention-output relative error versus scan bits per token. Ours is the curve, flat budgets and per-layer plans, in the basis the rule of §3.5 selects; baselines are points at their bit cost. (a) Qwen3-8B, 16k, $k = 256$. (b) Qwen3-8B, 32k, $k = 512$. (c) Qwen2.5-7B-Instruct-1M, 128k, $k = 2048$.

Table 6: Bits per token at which our scan reaches the error of the two 136-bit scans, on 7 settings. “plan” is the cheapest of the flat budget and the per-layer plan that reaches the target; basis follows the rule of §3.5.

Model	ctx/ k	basis	error	Double Sparsity $c=32$ (136 b)				SparQ $r=32$ (136 b)			
				Fathom bits	Fathom error	plan	error	Fathom bits	Fathom error	plan	
Qwen3-8B	16k/256	raw	0.0037	47	0.0033	per-layer	0.0022	56	0.0021	per-layer	
Qwen3-8B	32k/512	raw	0.0037	74	0.0024	flat	0.0012	92	0.0010	flat	
Qwen3-4B	16k/256	raw	0.0030	46	0.0027	per-layer	0.0022	56	0.0019	per-layer	
Llama-3.1-8B	4k/128	KLT	0.0015	47	0.0013	flat	0.0016	38	0.0015	per-layer	
Qwen2.5-7B	32k/512	KLT	0.0043	74	0.0025	flat	0.0025	74	0.0025	flat	
Qwen2.5-7B-1M	32k/512	KLT	0.0042	56	0.0038	flat	0.0020	74	0.0019	flat	
Qwen2.5-7B-1M	128k/2048	KLT	0.0024	47	0.0019	flat	0.0011	56	0.0011	per-layer	

Table 7: RULER-style tasks: mean score over tasks $\pm$ the standard error of the pooled per-sample scores. Bits as in §5.3. “Fathom” rows are raw planes on Qwen3-8B and “Fathom KLT” rows the rotated planes the basis rule selects for Qwen2.5-7B-1M. Per-task scores in Appendix B.

method	Qwen3-8B 32k, $k=128$		Qwen2.5-7B-1M 128k, $k=128$	
	bits	score	bits	score
dense	–	0.884 ± 0.015	–	0.677 ± 0.033
exact top- k oracle	–	0.874 ± 0.014	–	0.660 ± 0.033
Fathom 48	56	0.881 ± 0.014	–	–
Fathom 64	74	0.878 ± 0.014	–	–
Fathom KLT 48	–	–	56	0.671 ± 0.032
Fathom KLT 64	–	–	74	0.676 ± 0.035
Double Sparsity $c=32$	136	0.880 ± 0.014	136	0.655 ± 0.034
SparQ $r=16$	68	0.882 ± 0.014	68	0.644 ± 0.034
SparQ $r=32$	136	0.881 ± 0.014	136	0.640 ± 0.034
Loki $r=64$	272	0.882 ± 0.014	272	0.655 ± 0.033
2-bit thumbnail	288	0.877 ± 0.014	288	0.685 ± 0.035
block landmark	256	0.831 ± 0.016	256	0.531 ± 0.038

5.6 Real agent sessions

RULER-style tasks share structure with agent sessions, not content. Table 8 therefore decodes real ones: tra-

jectories of the OpenHands agent [19] on SWE-rebench issues [2], taken as recorded (system prompt, task, tool calls, file contents and command output), concatenated

Table 8: Long coding-agent sessions: 40 sessions of about 95k tokens built from real OpenHands trajectories on one repository each; every method greedily decodes the agent’s next step from the same cache (Qwen2.5-7B-Instruct-1M, $k = 512$). Step agreement is the word-level sequence-match ratio between a method’s step and the exact top- k step, with its standard error over sessions. Bold marks Fathom.

method	bits	step agreement with exact top- k , mean $\pm$ s.e.
exact top- k oracle	–	1.00 ± 0.00
Fathom 40	47	0.49 ± 0.04
Fathom 48	56	0.53 ± 0.04
Fathom 64	74	0.54 ± 0.04
Fathom 80	92	0.60 ± 0.05
Double Sparsity $c=32$	136	0.55 ± 0.04
SparQ $r=16$	68	0.49 ± 0.04
SparQ $r=32$	136	0.60 ± 0.04
block landmark	256	0.42 ± 0.03

per repository until the session holds 80k–100k tokens, with the agent’s next recorded step as the target. Each method decodes that step from the same prefill. The baseline is exact top- k decoding with the same k , which is what every scan is built to reproduce, and the score is step agreement: the word-level sequence-match ratio between a method’s step and the exact top- k step.

At $k = 2048$ (2% of the context) Fathom at 56 bits agrees with the exact top- k step at 0.67 against 0.49 for SparQ $r = 16$ and 0.47 for the block landmark index. Paired per session the margin over SparQ $r = 16$ is $+0.18 \pm 0.05$ (14 of 20 sessions), at 18% fewer bytes and the same GPU time. At $k = 512$ (0.5% of the context) the most accurate scan is SparQ $r = 32$ at 136 bits, 0.60 (Table 8). Fathom reaches the same agreement at 92 bits, 0.60 (paired -0.00 ± 0.04 , 20 of 40 sessions), 32% fewer bytes; its 56- and 74-bit reads sit with Double Sparsity’s 136-bit scan and at or above SparQ $r = 16$ (0.53 against 0.49, paired $+0.05 \pm 0.05$), and the landmark index is below all of them. This is what the attention-error tables predict at this selection ratio: SparQ $r = 32$ ’s error falls between Fathom’s 74- and 92-bit reads at 0.4% and between its 56- and 74-bit reads at 1.6% (Table 16), so the budget at which Fathom matches it, and the byte saving that follows, move with k/n (38–92 bits across settings, §5.3). Fathom’s channel statistics are calibrated once on Wikitext; recalibrating them on held-out agent transcripts, or on the session’s own prefill keys with no offline data at all, raises step agreement by $+0.01$ to $+0.07$ (within 1.5 standard errors) and changes attention error on agent-transcript keys by under 10%, so the ordering against SparQ $r = 32$ is set by the byte budget, not by the calibration domain (Appendix D, B5). The run was planned at 100 sessions and stopped at 40 for budget; the $k = 2048$ subset has 20. End-to-end task success with test execution under each scan is not measured and is left to future work.

Table 9: The same sessions at $k = 2048$ (2% of the context), 20 sessions: with the larger budget the scans separate.

method	bits	step agreement with exact top- k , mean $\pm$ s.e.
exact top- k oracle	–	1.00 ± 0.00
Fathom 48	56	0.67 ± 0.06
SparQ $r=16$	68	0.49 ± 0.05
block landmark	256	0.47 ± 0.05

Table 10: A1: flat budget versus per-layer plan, error at means 48 and 64, in the basis the rule selects.

Model	ctx/ k	basis	mean 48		mean 64	
			flat	per-layer	flat	per-layer
Qwen3-8B	16k/256	raw	0.0052	0.0021	0.0024	0.0010
Qwen3-8B	32k/512	raw	0.0081	0.0074	0.0024	0.0021
Qwen3-8B	32k/128	raw	0.0393	0.0363	0.0170	0.0182
Qwen3-4B	16k/256	raw	0.0034	0.0019	0.0021	0.0007
Llama-3.1-8B	4k/128	KLT	0.0009	0.0010	0.0004	0.0009
Qwen2.5-7B	32k/512	KLT	0.0044	0.0063	0.0025	0.0044
Qwen2.5-7B	32k/128	KLT	0.0215	0.0236	0.0127	0.0141
Qwen2.5-7B-1M	32k/512	KLT	0.0038	0.0038	0.0019	0.0019
Qwen2.5-7B-1M	128k/128	KLT	0.0196	0.0185	0.0111	0.0103
Qwen2.5-7B-1M	128k/512	KLT	0.0048	0.0040	0.0025	0.0017
Qwen2.5-7B-1M	128k/2048	KLT	0.0013	0.0011	0.0006	0.0004

6 Ablations

Each ablation changes one design choice and holds the rest fixed. Unless noted, the metric is held-out attention-output error at matched scan bits.

A1. Per-layer versus flat budgets. Table 10 compares the flat budget with the greedy per-layer plan at means 48 and 64 on every setting. The plan helps on the Qwen3 models at 16k and less at 32k, hurts on Qwen2.5-7B and Llama-3.1-8B, and is neutral to better on Qwen2.5-7B-1M; a 16k plan evaluated at 32k on Qwen3-8B is worse than the flat budget (the “plan from 16k” rows of Table 14). Flat is the default. Figure 11 shows the plans.

A2. Basis. Table 11 and Figure 5 compare raw and KLT-rotated planes at the same budgets on every setting. The rotation raises error on Qwen3 and lowers it on the three models without QK-norm. QK-norm equalizes

Table 11: A2: raw versus KLT-rotated planes, error at means 48 and 64.

Model	ctx/ k	mean 48		mean 64	
		raw	KLT	raw	KLT
Qwen3-8B	16k/256	0.0052	0.0106	0.0024	0.0064
Qwen3-8B	32k/512	0.0081	0.2908	0.0024	0.1576
Qwen3-4B	16k/256	0.0034	0.0167	0.0021	0.0102
Llama-3.1-8B	4k/128	0.0025	0.0009	0.0012	0.0004
Qwen2.5-7B	32k/512	0.0075	0.0044	0.0043	0.0025
Qwen2.5-7B	32k/128	0.0264	0.0215	0.0153	0.0127
Qwen2.5-7B-1M	32k/512	0.0066	0.0038	0.0038	0.0019
Qwen2.5-7B-1M	128k/128	0.0300	0.0196	0.0162	0.0111
Qwen2.5-7B-1M	128k/512	0.0082	0.0048	0.0036	0.0025
Qwen2.5-7B-1M	128k/2048	0.0025	0.0013	0.0013	0.0006

channel scales and leaves the per-query sparsity in the raw basis; without it, the rotation concentrates it.

A3. Selection ratio. In Table 16 the crossing with the 136-bit scans moves to higher budgets as k/n falls from 1.6% to 0.1%, with raw and with rotated planes. The effect is common to every scan including the full 4-bit one.

A4. Uniform versus water-filled depth. The 2-bit thumbnail reads every channel at two planes, 288 bits per token. Its error in Tables 14 and 15 is reached by the water-filled read at 28–148 bits on all 7 settings (the 148-bit case is Qwen2.5-7B-1M at 128k with $k = 2048$, where the thumbnail is also strong). Uniform depth spends bits on channels that carry no score for the query.

A5. SparQ’s selection rule. SparQ’s grouped-query rule reads one channel set per KV head. Letting every query head pick its own r channels and reading the union lowers error but costs 2.1–3 $\times$ the bytes (Table 18, Appendix D). The $r = 16$ variant sits at 148–200 bits and has higher error than our 146-bit read on every setting (Tables 14 and 15); the $r = 32$ variant reads 284–373 bits, beyond any budget we measure.

A6. Measurement controls. Wall-clock exceeds GPU time by the time the GPU waits on the Python harness. At 1M the gap is 48 ms per step for our 56-bit read, 15 ms for the 74-bit read, 24 and 14 ms for the 16- and 32-channel scans, 22 ms for the landmark index and 29 ms for the thumbnail (Table 2). The two plane reads launch the same kernels yet show different gaps, which we did not isolate; the gap is host-side and absent from GPU time, and a fused implementation would not have it, which is why GPU time is the primary metric. The wall-clock ratios of every baseline are below the GPU-time ratios. Every timing row uses a fixed-size run list with no host synchronization, and the gather’s staging buffer was checked bit-exact against a direct GPU scan over the same keys on this pod before any

timing was recorded. Two engineering variants were tried and are not used: a chunked pipeline that overlaps the gather of chunk $i+1$ with the scan of chunk i on a second stream, bit-exact but no faster, because both gathers already run at the link rate and the scan kernel slows under contention; and a sweep of the Triton launch configurations of both scan kernels and a tensor-core variant, none faster than the configurations reported. Two-stage and reduced-precision top- k were no faster than the library top- k either.

7 Analysis

HBM-resident scan kernel. With everything in HBM the byte saving does not become time (Table 17, Fig. 9). The planes kernel extracts one bit per shift-and-mask and applies G multiply-adds per extracted bit, where a 4-bit nibble carries four times the information per extracted element, so it reaches a fraction of the bandwidth the nibble scans reach. The 32-channel scan’s kernel time is $0.73\times$ ours at 32k and $0.70\times$ at 128k; ours is slower in the kernel despite reading 38% fewer bytes on these layers. The 32-channel scan is 16–24% of its sparse-attention step and ours 21–30% (top- k , gather and exact attention are method-independent), so at the step level the per-token scans land within 6% of each other (45–48 ms at 128k) and ours is not the fastest, which is what the HBM-index columns of Table 3 show.

Arithmetic per byte. The scan’s arithmetic is one multiply-add per (active channel, token, head), so reading fewer planes of a channel removes bytes, not multiply-adds. Bit extraction adds one to two integer operations per bit read, four times the per-bit cost of a nibble scan. Counting INT32 lanes times clock against HBM bandwidth (108 SMs, 64 lanes, 1.41 GHz, 2.0 TB/s), an A100 offers about 5 integer operations per byte, an H100 SXM about the same and a bandwidth-poor L4 about 25, so a scan that is arithmetic-bound on the A100 is arithmetic-bound on any current data-centre GPU; the HBM-resident behaviour above is a property of the operation, not of the card. Amdahl’s law compounds it: where the scan is a minority of sparse attention, even a zero-time scan yields little. Over PCIe the ordering reverses. Over PCIe every method runs at the link’s rate, the arithmetic is idle, and the byte ratio approaches the time ratio (Table 2).

Access pattern. The Triton gather of contiguous runs from pinned host memory moves 25.1–26.2 GB/s with the best block size at every run size from 4 KB (one plane of one channel at 32k) to 512 KB (four planes at 1M), and 23.7 GB/s at 4 KB with the worst. That is the rate of one `cudaMemcpyAsync` of the same bytes (21.6–25.9 GB/s), and runs placed at $4\times$ stride lose nothing

(Fig. 10). One copy call per run reaches 0.4 GB/s at 4 KB runs, about 11 μ s per call, and 22.4 GB/s only at 512 KB. The bit-plane layout is what makes a per-query selection of planes a set of long contiguous runs, and the gather kernel moves those runs at link rate even at 4 KB granularity; a landmark or label-cache index is one contiguous block and needs no gather, which is why every baseline in Table 2 is given the same transfer and the comparison is one of bytes.

Memory. The store has a memory cost. As a separate index it is 68 bytes per token per KV head, four times Double Sparsity’s 17 and twice a 32-byte landmark index; its advantage is in bytes read per query, not bytes stored. The store becomes free only in a stack whose K cache is already 4-bit and channel-major, where the winners’ keys are reconstructed from all four planes; we have not built or measured that path, and reconstructing single tokens from a channel-major layout is not cheap.

Sensitivity of the downstream tasks. On RULER-style tasks at $k \geq 128$ the top- k set is robust to scan error of the magnitude all per-token scans exhibit; those tasks separate selection granularity (blocks versus tokens) and the top- k budget itself, not the scans. Real agent steps are more sensitive: at a 2% budget the scan’s fidelity to the exact top- k step is visible in what the agent writes (§5.6). A task that separated 0.002 from 0.004 attention error would need a much smaller k or a much longer dependency chain than RULER’s.

8 Limitations

The systems regime this paper targets, offloaded caches and million-token contexts with many concurrent sessions, is measured directly. Quality is measured on attention-output error, on synthetic retrieval and tracking tasks, and on real agent sessions scored by step agreement with exact top- k decoding (40 of 100 planned sessions at $k = 512$, 20 at $k = 2048$); end-to-end task success with test execution under each scan is not measured and is left to future work. With the scan index resident in HBM the method is not faster than the 4-bit channel scans; its time advantage requires the index to live in a slower tier. The landmark baseline is a reimplement of ShadowKV’s chunk means in a setting (host-resident index, the paper’s k) that ShadowKV and Quest do not use, and CPU-side retrieval of the RetroInfer and MagicPIG kind is not compared. Timing beyond 128k used synthetic KV contents. The Triton kernels are prototypes and the offload harness pays a per-method Python issue cost that a fused implementation would not; GPU time is reported for that reason. Per-layer plans must be calibrated at the deployment context length or replaced by a flat budget. Channel statistics are calibrated offline; on agent transcripts the domain of that

calibration moves step agreement by at most +0.07 and the session-calibrated variant removes the dependence (Appendix D, B5), but domains further from both were not tested. Fidelity is measured on one held-out window per setting, and RULER-style scores carry standard errors of a few points, so differences between per-token scans smaller than that are not established. The offload tables are batch 1, with one batch-2 check; the many-session case that motivates the regime is argued from index sizes rather than measured. The instruct model at 128k was prompted without its chat template, which lowers its absolute scores without affecting the comparison between scans. Top- k is selected per query head for every method; SparQ’s group-shared top- k , which would reduce the winner rows for SparQ and for any other method, and its mean-value reallocation, which would lower SparQ’s absolute error without changing the comparison of scans, are not evaluated.

9 Conclusion

Fathom is a key scan for sparse decoding when the KV cache and its index live in host memory. There, a decode step at one million tokens runs $1.67\times$ faster in GPU time than with the 136-bit scans and $2.50\times$ faster than with a landmark index, and against SparQ’s 68-bit read it moves 18% fewer bytes at the same GPU time with $1.1\text{--}5.3\times$ lower attention error, and at 47 bits it is $1.11\times$ faster. The mechanism is a 4-bit K cache stored in channel-major bit planes, so a prefix read is an exact lower-precision quantizer, and a per-query water-filling rule that allocates bits to the channels that carry the query’s score variance. At Double Sparsity’s error it reads 46–74 bits where fixed-depth scans read 136, on seven settings up to 128k; it matches the exact top- k oracle on RULER-style tasks and, on real coding-agent sessions at a 2% budget, agrees with the exact top- k step at 0.67 against 0.49 for SparQ $r = 16$. Where everything sits in HBM the method is not faster, because the scan is arithmetic-bound there and reading fewer bits does not remove multiply-adds; a fused kernel would narrow the gap but not remove the arithmetic bound. The method fits serving stacks that already keep a 4-bit K cache and hold it in a slower tier.

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [2] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik,

- Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [3] Yaoqi Chen, Jinkai Zhang, Baotong Lu, Qianxi Zhang, Chengruidong Zhang, Jing Liu, Jingjia Luo, Di Liu, Huiqiang Jiang, Qi Chen, Bailu Ding, Xiao Yan, Jiawei Jiang, Chen Chen, Mingxing Zhang, Cheng Li, Yuqing Yang, Fan Yang, and Mao Yang. Retroinfer: A vector storage engine for scalable long-context llm inference. *Proceedings of the VLDB Endowment*, 19(5):1016–1031, 2026.
- [4] Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Léon Bottou, Zhihao Jia, and Beidi Chen. Magicpig: Lsh sampling for efficient llm generation. In *International Conference on Learning Representations (ICLR)*, 2025.
- [5] Allen Gersho and Robert M. Gray. *Vector Quantization and Signal Compression*. Kluwer Academic Publishers, 1992.
- [6] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [7] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesht, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*, 2024.
- [8] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. Infinigen: Efficient generative inference of large language models with dynamic kv cache management. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [9] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. Retrievalattention: Accelerating long-context llm inference via vector retrieval. *arXiv preprint arXiv:2409.10516*, 2024.
- [10] Zhigeng Liu, Zhiyuan Ning, Ruixiao Li, Xiaoran Liu, Yuerong Song, Min Zhang, Ziwei He, and Xipeng Qiu. Faster than flash: Exploiting attention sparsity for efficient long-context decoding. In *International Conference on Machine Learning (ICML)*, 2026.
- [11] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. In *International Conference on Machine Learning (ICML)*, 2024.
- [12] Llama Team, AI @ Meta. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [13] Yeonhong Park, Jake Hyun, Sanglyul Cho, Bonggeun Sim, and Jae W. Lee. Any-precision llm: Low-cost deployment of multiple, different-sized llms. In *International Conference on Machine Learning (ICML)*, 2024.
- [14] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [15] Luka Ribar, Ivan Chelombiev, Luke Hudlass-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. Sparq attention: Bandwidth-efficient llm inference. In *International Conference on Machine Learning (ICML)*, 2024.
- [16] Prajwal Singhania, Siddharth Singh, Shwai He, Soheil Feizi, and Abhinav Bhatele. Loki: Low-rank keys for efficient sparse attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [17] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. Shadowkv: Kv cache in shadows for high-throughput long-context llm inference. In *International Conference on Machine Learning (ICML)*, 2025.
- [18] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. In *International Conference on Machine Learning (ICML)*, 2024.
- [19] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.
- [20] Chaojun Xiao, Pengl Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. Infilm: Training-free long-context extrapolation for llms with an efficient context memory. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

- [21] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- [22] Shuo Yang, Ying Sheng, Joseph E. Gonzalez, Ion Stoica, and Lianmin Zheng. Post-training sparse attention with double sparsity. *arXiv preprint arXiv:2408.07092*, 2024.
- [23] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. Turboquant: Online vector quantization with near-optimal distortion rate. In *International Conference on Learning Representations (ICLR)*, 2026.
- [24] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H₂o: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A Worked example

Six keys, four channels, two query heads sharing the KV head. Keys (rows $t_0 \dots t_5$) and their channel variances:

$$K = \begin{pmatrix} 1.8 & 0.4 & -0.2 & 0.05 \\ -1.1 & 0.9 & 0.3 & -0.02 \\ 0.3 & -1.2 & 0.1 & 0.04 \\ 1.2 & 0.7 & 0.4 & 0.01 \\ -0.6 & -0.3 & -0.5 & 0.03 \\ 0.9 & 1.1 & -0.3 & -0.05 \end{pmatrix}, \quad \text{Var} = \begin{pmatrix} 1.222 \\ 0.755 \\ 0.127 \\ 0.001 \end{pmatrix}^\top.$$

The queries are $(3, 1, 2, 0.1)$ for head A and $(1, 2, -1, 0.2)$ for head B. Exact scores give top-2 sets $\{t_0, t_3\}$ for A and $\{t_5, t_0\}$ for B; B’s decision between t_0 and t_3 rests on channel 2, where t_3 has 0.4 and t_0 has -0.2 .

Importance weights. Eq. 3 sums the squared query weights of both heads against the channel variances: $g_0 = (3^2 + 1^2) 1.222 = 12.22$, $g_1 = (1^2 + 2^2) 0.755 = 3.77$, $g_2 = (2^2 + 1^2) 0.127 = 0.63$, $g_3 = (0.1^2 + 0.2^2) 0.001 \approx 0$.

Finding the water line. θ is a free parameter. For any value of θ , Eq. 4 turns the four importances into four depths, and the depths have a sum. A small θ makes every g_j/θ large and hands out many bits; a large θ hands out few. The water line is the smallest θ whose depths still fit the budget B , because fewer bits than the budget would waste budget and more would exceed it. Bisection finds it by trial. Take $B = 8$ and start from the bracket $[0.001, 10]$ on θ . Each trial is the midpoint of the current bracket on a log scale, that is the geometric mean of its two ends ($\sqrt{0.001 \times 10} = 0.1$, then $\sqrt{0.001 \times 0.1} = 0.01$, and so on); if the trial’s depths

overflow B it becomes the new lower end lo, otherwise the new upper end hi:

[lo, hi]	trial θ	$\log_4(g_j/\theta)$	t	sum	
[0.001, 10]	0.1	(3.47, 2.62, 1.33, -5.5)	(3, 3, 1, 0)	7	fits, hi $\leftarrow 0.1$
[0.001, 0.1]	0.01	(5.13, 4.28, 2.99, -3.8)	(4, 4, 3, 0)	11	over, lo $\leftarrow 0.01$
[0.01, 0.1]	0.0316	(4.30, 3.45, 2.16, -4.7)	(4, 3, 2, 0)	9	over, lo $\leftarrow 0.0316$
[0.0316, 0.1]	0.0562	(3.88, 3.03, 1.74, -5.1)	(4, 3, 2, 0)	9	over, lo $\leftarrow 0.0562$
[0.0562, 0.1]	0.075	(3.67, 2.83, 1.54, -5.3)	(4, 3, 2, 0)	9	over, lo $\leftarrow 0.075$
[0.075, 0.1]	0.0866	(3.57, 2.72, 1.43, -5.4)	(4, 3, 1, 0)	8	fits, hi $\leftarrow 0.0866$
[0.075, 0.0866]	0.0806	(3.62, 2.77, 1.48, -5.3)	(4, 3, 1, 0)	8	fits, hi $\leftarrow 0.0806$
[0.075, 0.0806]	0.0777	(3.65, 2.80, 1.51, -5.3)	(4, 3, 2, 0)	9	over, lo $\leftarrow 0.0777$

After 30 halvings the upper end has settled at $\theta = 0.079$, and the depths there, $t = (4, 3, 1, 0)$, use all 8 bits. Channel 2 receives one plane while $\text{round}(\log_4(0.63/\theta)) = 1$, that is while $0.5 \leq \log_4(0.63/\theta) < 1.5$, which is $0.63/4^{1.5} < \theta \leq 0.63/4^{0.5}$, or $0.0788 < \theta \leq 0.315$; just below 0.0788 channel 2 would take a second plane and the sum would become 9. So the water line for $B = 8$ sits right at the point where the next bit anywhere would overflow the budget. The same procedure for $B = 6$ stops at $\theta = 0.118$, where $\log_4(g_j/\theta) = (3.35, 2.50, 1.21, -5.3)$ and $t = (3, 2, 1, 0)$; below 0.1178 = $3.77/4^{2.5}$ channel 1 would take a third plane. Channel 3 is never read at either budget; channel 2 gets one plane at both because its importance is a factor of six below channel 1’s, and one plane costs one bit.

One plane of channel 2. The block absmax of channel 2 is $a_2 = 0.5$, so the cell width is $s = a_2/8 = 0.0625$ and the 4-bit codes of the six keys are $(4, 12, 9, 14, 0, 3)$, in binary 0100, 1100, 1001, 1110, 0000, 0011. The first plane is the sign bit. Reading it alone, $c^{(1)} = c \gg 3 \in \{0, 1\}$, and the dequantized value $(c^{(1)} + \frac{1}{2} - 1)a_2$ is -0.25 for t_0, t_4, t_5 and $+0.25$ for t_1, t_2, t_3 ; the true values are $-0.2, 0.3, 0.1, 0.4, -0.5, -0.3$. One bit already places t_3 above t_0 on this channel. With depths $(4, 3, 1, 0)$ head B’s approximate scores are $(2.84, 0.24, -2.01, 2.49, -0.61, 3.36)$ against exact $(2.81, 0.40, -2.19, 2.20, -0.69, 3.39)$, so B selects $\{t_5, t_0\}$ correctly; head A’s are $(5.01, -1.79, 0.46, 4.96, -2.34, 3.59)$ against $(5.40, -1.80, -0.10, 5.10, -3.10, 3.19)$ and A selects $\{t_0, t_3\}$.

The other methods at 8 bits. Loki with $r = 2$ keeps directions that are almost exactly channels 0 and 1 and ranks B as $\{t_5, t_3\}$. Double Sparsity with $c = 2$ picks channels 0 and 1 offline and makes the same mistake. SparQ with $r = 2$ under its grouped-query rule sums $|q|$ over the two heads, $(4, 3, 3, 0.3)$, picks channels 0 and 1 (channel 1 wins the tie with channel 2 by index), reads 8 bits and ranks B as $\{t_5, t_3\}$. A 2-bit thumbnail (8 bits) reads all four channels at two planes and mis-ranks both heads. At $B = 6$ Fathom’s depths $(3, 2, 1, 0)$ also mis-rank B, which is the budget at which the method fails on this example. Figure 4 draws the read shapes.

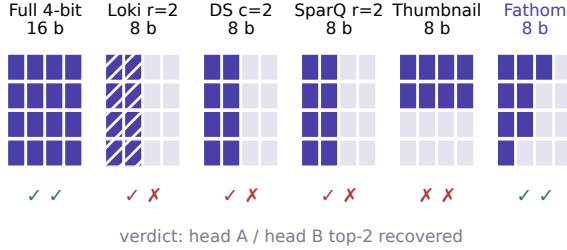


Figure 4: Bit-cells read per token by each method on the worked example (columns: channels, rows: planes). Checks mark heads ranked correctly.

B RULER-style task details

The haystack is Wikitext-103 test text tokenized once; each sample takes a random window and inserts needles at random depths in $[0.05, 0.95]$ of the window. Needle templates follow RULER, for example “One of the special magic numbers for {key} is: {7 digits}.” with one target key (single), one target plus seven decoy keys (multi-key; RULER’s multikey-1 default is three decoys), four values for one key (multi-value), or four keys each with one value (multi-query). Variable tracking chains four assignments through one decoy chain; frequent-word extraction asks for the three most frequent synthetic words drawn with Zipf exponent 2. Generation lengths are 12 tokens for single numbers, 40 for lists and 24 for words; the score is the fraction of gold strings contained in the greedy generation. Prompts are plain completions without the chat template.

C Additional tables and figures

The tables and figures referenced from the main text but not essential to its argument.

D Additional ablations

B1. Store precision. With a per-query depth read the offline allocation of the store hardly matters (Table 19). We use the uniform 4-bit store, which is a 4-bit copy of K and needs no allocation step.

B2. Loki’s rank. Loki’s failure on Qwen3-8B is rank, not quantization: fp16 coordinates at $r = 32$ do not repair it and $r = 64$ at 4 bits does (Table 20). On Llama-3.1-8B at 4k $r = 32$ is fine. The failure is model- and length-specific, which is why the equal-error references are Double Sparsity and SparQ $r = 32$.

B3. Active channels. Table 21 gives the measured number of channels a read touches at each budget.

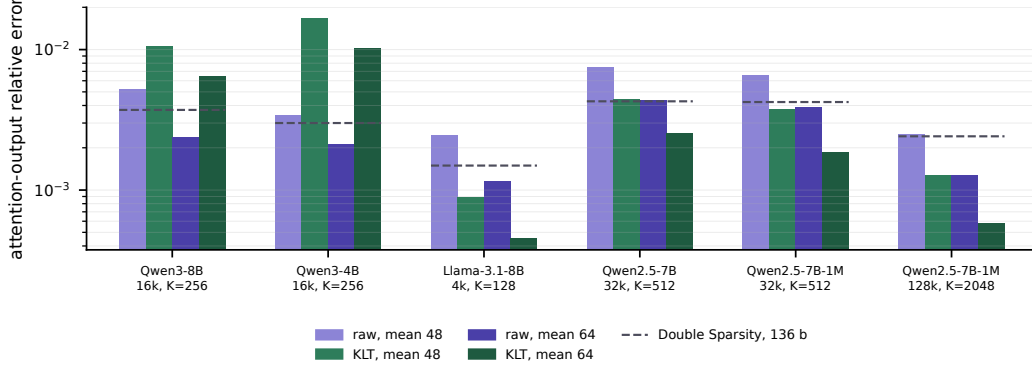
B5. Calibration domain. Fathom’s variances and KLT bases, and Double Sparsity’s channel order, are calibrated on Wikitext-103; SparQ needs no calibration. Table 22 measures attention error on 128k tokens of held-out agent transcripts (rendered exactly as the sessions of §5.6, from repositories not used there) with the statistics taken from Wikitext and from a second set of agent transcripts. Matching the domain lowers Fathom’s error by at most 0.0057 to 0.0053 at 56 bits and 0.0017 to 0.0014 at 92 bits, and Double Sparsity’s by 0.0050 to 0.0044; no rank against SparQ $r = 32$ changes. Table 23 repeats the $k = 512$ agent decode with the agent-domain statistics and with a third option that removes offline calibration altogether: the mean, eigenbasis and eigenvalues of each layer’s keys computed from the session’s own prefill cache (one covariance per KV head, under a millisecond at 100k tokens). Both raise step agreement by +0.01 to +0.07 over the Wikitext calibration, within 1.5 standard errors, and the session-calibrated 92-bit read at 0.64 is $+0.04 \pm 0.04$ against SparQ $r = 32$; recalibrating Double Sparsity changes it by -0.01. The exact top- k step was reproduced verbatim in all 40 sessions between the two runs.

E Hardware and software

A100-SXM4-80GB (RunPod secure cloud, PCIe 4.0 host link, 2 TB host RAM, 16 vCPU), PyTorch 2.8, Triton 3.4, Transformers 4.56.1, CUDA 12.8. Llama-3.1-8B activations from an NVIDIA L4 (24 GB). Kernels are Triton; the gather kernel copies contiguous runs from mapped pinned host memory into HBM staging. Every table and every number in the text is generated from the result files by one script.

Table 12: Qwen3-8B, 32k, $k = 128$, 40 samples per task.

method	niah_single	niah_multikey	niah_multivalue	niah_multiquery	vt	fwe
dense	1.000	0.975	1.000	0.950	0.485	0.892
exact top- k oracle	1.000	0.975	1.000	0.938	0.525	0.808
Fathom 48	1.000	0.975	1.000	0.931	0.565	0.817
Fathom 64	1.000	0.975	1.000	0.931	0.535	0.825
Double Sparsity $c=32$	1.000	0.975	1.000	0.944	0.560	0.800
SparQ $r=16$	1.000	0.975	1.000	0.944	0.575	0.800
SparQ $r=32$	1.000	0.975	1.000	0.938	0.555	0.817
Loki $r=64$	1.000	0.975	1.000	0.938	0.545	0.833
2-bit thumbnail	1.000	0.975	1.000	0.938	0.535	0.817
block landmark	0.975	0.925	0.925	0.850	0.505	0.808

**Figure 5:** Raw versus rotated planes at means 48 and 64 on six settings (Table 11 has all of them). Ticks mark Double Sparsity at 136 bits.**Table 13:** Qwen2.5-7B-Instruct-1M, 128k, $k = 128$, 20 samples per task.

method	niah_multikey	niah_multiquery	vt	fwe
dense	1.000	0.375	0.550	0.783
exact top- k oracle	1.000	0.388	0.520	0.733
Fathom KLT 48	1.000	0.375	0.560	0.750
Fathom KLT 64	1.000	0.400	0.520	0.783
Double Sparsity $c=32$	1.000	0.375	0.530	0.717
SparQ $r=16$	1.000	0.375	0.500	0.700
SparQ $r=32$	1.000	0.400	0.510	0.650
Loki $r=64$	1.000	0.400	0.520	0.700
2-bit thumbnail	1.000	0.400	0.540	0.800
block landmark	0.700	0.263	0.460	0.700

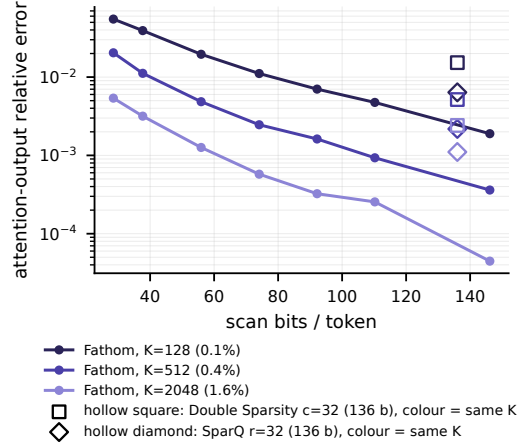
**Figure 6:** Qwen2.5-7B-Instruct-1M at 128k: error versus bits at three selection ratios. Hollow markers are Double Sparsity (square) and SparQ $r = 32$ (diamond) at 136 bits in the colour of the same k .

Table 14: Qwen3-8B, all methods, bits and attention-output error. Raw planes; flat budgets and per-layer plans, including the 16k plan evaluated at 32k.

	Qwen3-8B 16k/ $K=256$		Qwen3-8B 32k/ $K=512$	
	bits	error	bits	error
Fathom flat 40	47	0.0075	46	0.0142
Fathom flat 48	56	0.0052	56	0.0081
Fathom flat 64	74	0.0024	74	0.0024
Fathom flat 80	92	0.0012	92	0.0010
Fathom flat 128	146	0.0003	146	0.0001
Fathom per-layer 40	47	0.0033	46	0.0091
Fathom per-layer 48	56	0.0021	55	0.0074
Fathom per-layer 64	74	0.0010	74	0.0021
Fathom per-layer 48, plan from 16k	–	–	56	0.0103
Fathom per-layer 64, plan from 16k	–	–	74	0.0035
SparQ $r=16$	68	0.0077	68	0.0080
SparQ $r=32$	136	0.0022	136	0.0012
Double Sparsity $c=32$	136	0.0037	136	0.0037
Loki $r=32$	136	0.0109	136	0.4033
Loki $r=64$	272	0.0004	272	0.0008
2-bit thumbnail	288	0.0028	288	0.0102
full 4-bit scan	544	<0.0001	544	<0.0001

Table 15: The other four headline settings. “Fathom flat” rows use raw planes, “Fathom KLT” rows the rotated planes the basis rule selects for these models.

	Qwen3-4B 16k/ $K=256$		Llama-3.1-8B 4k/ $K=128$		Qwen2.5-7B 32k/ $K=512$		Qwen2.5-7B-1M 32k/ $K=512$	
	bits	error	bits	error	bits	error	bits	error
Fathom flat 40	46	0.0050	48	0.0033	46	0.0107	47	0.0098
Fathom flat 48	56	0.0034	57	0.0025	55	0.0075	56	0.0066
Fathom flat 64	74	0.0021	75	0.0012	73	0.0043	74	0.0038
Fathom flat 80	92	0.0012	93	0.0007	91	0.0025	92	0.0026
Fathom flat 128	146	0.0003	148	0.0002	146	0.0010	146	0.0013
SparQ $r=16$	68	0.0071	68	0.0044	68	0.0090	68	0.0091
SparQ $r=32$	136	0.0022	136	0.0016	136	0.0025	136	0.0020
Double Sparsity $c=32$	136	0.0030	136	0.0015	136	0.0043	136	0.0042
Loki $r=32$	136	0.0155	136	0.0008	136	0.0026	136	0.0024
Loki $r=64$	272	0.0004	272	0.0001	272	0.0002	272	0.0002
2-bit thumbnail	288	0.0356	288	0.0002	288	0.0025	288	0.0014
full 4-bit scan	544	<0.0001	544	<0.0001	544	<0.0001	544	<0.0001
Fathom KLT 40	47	0.0214	47	0.0013	47	0.0065	47	0.0052
Fathom KLT 48	56	0.0167	57	0.0009	56	0.0044	56	0.0038
Fathom KLT 64	74	0.0102	75	0.0004	74	0.0025	74	0.0019
Fathom KLT 80	93	0.0060	93	0.0003	92	0.0017	92	0.0010

Table 16: Qwen2.5-7B-Instruct-1M, 128k context. Error at three selection ratios k/n (0.1%, 0.4%, 1.6%).

	Qwen2.5-7B-1M 128k/ $K=128$		Qwen2.5-7B-1M 128k/ $K=512$		Qwen2.5-7B-1M 128k/ $K=2048$	
	bits	error	bits	error	bits	error
Fathom KLT 48	56	0.0196	56	0.0048	56	0.0013
Fathom KLT 64	74	0.0111	74	0.0025	74	0.0006
Fathom KLT 80	92	0.0070	92	0.0016	92	0.0003
Fathom raw 48	55	0.0300	55	0.0082	55	0.0025
Fathom raw 64	73	0.0162	73	0.0036	73	0.0013
SparQ $r=16$	68	0.0303	68	0.0116	68	0.0055
SparQ $r=32$	136	0.0064	136	0.0022	136	0.0011
Double Sparsity	136	0.0153	136	0.0052	136	0.0024
Loki $r=32$	136	0.0132	136	0.0033	136	0.0013
Loki $r=64$	272	0.0015	272	0.0004	272	0.0001
2-bit thumbnail	288	0.0131	288	0.0022	288	0.0001
full 4-bit scan	544	0.0002	544	<0.0001	544	<0.0001

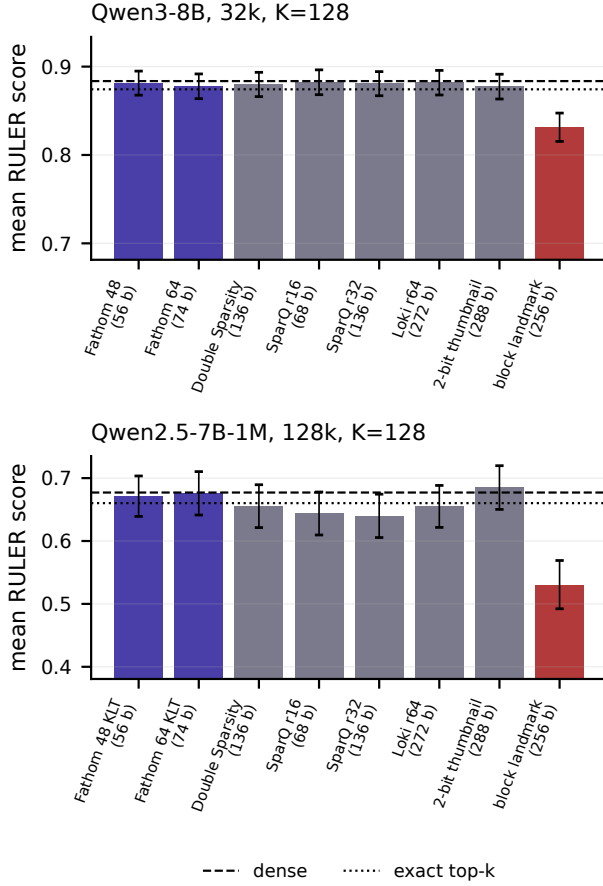


Figure 7: RULER-style mean score per method with bits in parentheses. Dashed: dense; dotted: exact top-k oracle. (a) Qwen3-8B, 32k, $k = 128$. (b) Qwen2.5-7B-Instruct-1M, 128k, $k = 128$.

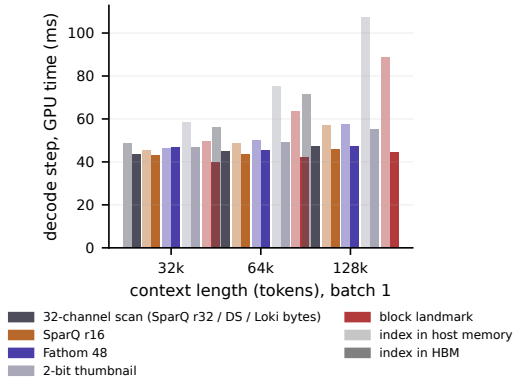


Figure 8: Real prefill at 32k–128k: GPU time per step with the scan index in host memory versus in HBM. With every index resident in HBM ours is not faster than the channel scans or the landmark index.

Table 17: Triton scan kernels on the A100 with the stores in HBM: Qwen3-8B, batch 4×8 KV heads, $k = 512$, layers 0, 1, 2, 18 and 30, CUDA-graph replays, median of 30. Ours uses the per-layer plan, so its bits are the mean over these five layers, which include the early layers with the largest budgets. Scan time per layer and achieved HBM bandwidth.

method	32k tokens			128k tokens		
	bits	scan ms	GB/s	bits	scan ms	GB/s
Fathom 48	84	0.140	75	84	0.415	102
Fathom 64	95	0.155	77	95	0.457	106
SparQ $r=16$ (16 channels)	68	0.060	148	68	0.164	218
32-channel scan	136	0.102	175	136	0.291	245
full 4-bit scan	544	0.330	217	544	1.010	282

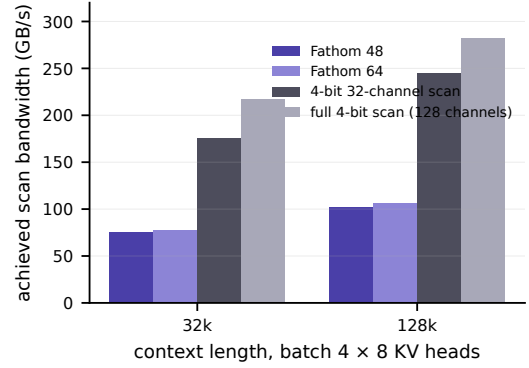


Figure 9: Achieved HBM bandwidth of the A100 scan kernels at 32k and 128k tokens, batch 4. The planes kernel reaches a third to a half of the channel scans’ bandwidth; none of them approaches the 2 TB/s peak.

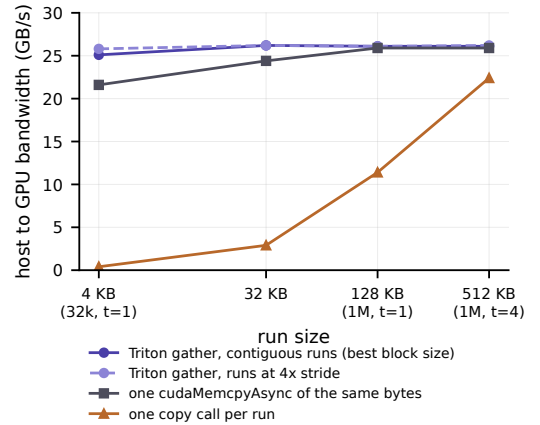


Figure 10: PCIe transfer bandwidth versus run size on the A100 host: the Triton gather of contiguous runs, a single memcopy of the same bytes, and per-run copy calls.

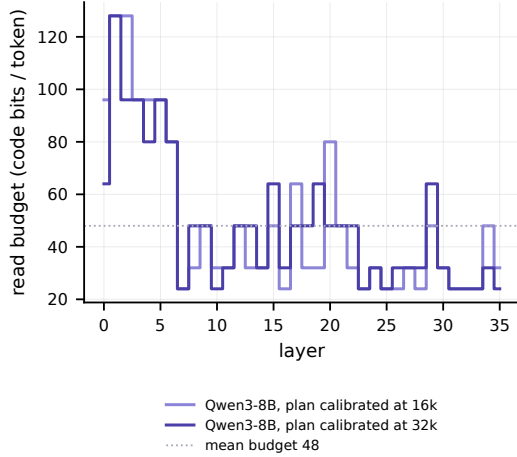


Figure 11: Per-layer bit budgets (code bits, before scales) for Qwen3-8B at mean 48, calibrated at 16k and at 32k.

Table 18: B4: SparQ under its published grouped-query rule (one channel set per KV head) and a per-head variant that reads the union of the heads’ picks.

Model	ctx/k	r=16				r=32			
		published rule	error	per-head variant		published rule	error	per-head variant	
Qwen3-8B	16k/256	68	0.0077	156	0.0037	136	0.0022	298	0.0008
Qwen3-8B	32k/512	68	0.0080	156	0.0041	136	0.0012	296	0.0002
Qwen3-8B	32k/128	68	0.0305	156	0.0228	136	0.0073	296	0.0021
Qwen3-4B	16k/256	68	0.0071	156	0.0043	136	0.0022	299	0.0011
Llama-3.1-8B	4k/128	68	0.0044	148	0.0041	136	0.0016	284	0.0009
Qwen2.5-7B	32k/512	68	0.0090	200	0.0030	136	0.0025	373	0.0010
Qwen2.5-7B	32k/128	68	0.0238	200	0.0097	136	0.0068	373	0.0024
Qwen2.5-7B-1M	32k/512	68	0.0091	197	0.0024	136	0.0020	366	0.0002
Qwen2.5-7B-1M	128k/128	68	0.0303	192	0.0170	136	0.0064	362	0.0020
Qwen2.5-7B-1M	128k/512	68	0.0116	192	0.0054	136	0.0022	362	0.0008
Qwen2.5-7B-1M	128k/2048	68	0.0055	192	0.0020	136	0.0011	362	0.0003

Table 19: B1: Qwen3-8B, 16k, $k = 256$. The same read budgets over a uniform 4-bit store, a uniform 8-bit store and a rate-allocated 128-bit store.

read budget	4-bit store		8-bit store		rate-allocated store	
	bits	error	bits	error	bits	error
mean 48	56	0.0052	56	0.0052	56	0.0054
mean 64	74	0.0024	74	0.0024	74	0.0026
mean 80	92	0.0012	92	0.0012	92	0.0017

Table 20: B2: Loki at rank 32 in fp16 and 4-bit, and at rank 64.

Model	ctx/k	r=32 fp16 (512 b)	r=32 4-bit (136 b)	r=64 4-bit (272 b)
Qwen3-8B	16k/256	0.0105	0.0109	0.0004
Qwen3-8B	32k/512	0.4007	0.4033	0.0008
Qwen3-8B	32k/128	0.6749	0.6898	0.0110
Llama-3.1-8B	4k/128	0.0008	0.0008	0.0001

Table 21: B3: mean number of active channels (of 128) per query read, flat budgets, headline settings.

Model	ctx/k	mean 32	mean 48	mean 64	mean 80	mean 128
Qwen3-8B	16k/256	22	31	40	48	73
Qwen3-8B	32k/512	21	30	39	48	73
Qwen3-4B	16k/256	21	31	40	49	74
Llama-3.1-8B	4k/128	24	34	44	54	79
Qwen2.5-7B	32k/512	22	31	40	49	74
Qwen2.5-7B-1M	32k/512	23	32	40	49	74
Qwen2.5-7B-1M	128k/2048	23	32	40	49	72

Table 22: B5: attention error on agent-transcript keys (Qwen2.5-7B-Instruct-1M, 128k) with statistics calibrated on Wikitext or on held-out agent transcripts. SparQ has no calibration.

method	bits	k=512 (0.4%)		k=2048 (1.6%)	
		Wikitext calib.	agent calib.	Wikitext calib.	agent calib.
Fathom KLT 48	56	0.0057	0.0053	0.0018	0.0015
Fathom KLT 64	74	0.0030	0.0030	0.0008	0.0007
Fathom KLT 80	92	0.0017	0.0014	0.0003	0.0002
SparQ $r=16$	68	0.0102	0.0102	0.0057	0.0057
SparQ $r=32$	136	0.0025	0.0025	0.0011	0.0011
Double Sparsity	136	0.0050	0.0044	0.0023	0.0021

Table 23: B5: the $k = 512$ agent sessions of Table 8 with Fathom and Double Sparsity calibrated on Wikitext (as in the main text), on held-out agent transcripts, or on the session’s own keys.

method	bits	step agreement with exact top-k, mean $\pm$ s.e.
Fathom 48	56	0.53 $\pm$ 0.04
Fathom 48, agent-calibrated	56	0.54 $\pm$ 0.04
Fathom 48, session-calibrated	56	0.57 $\pm$ 0.04
Fathom 64	74	0.54 $\pm$ 0.04
Fathom 64, agent-calibrated	74	0.60 $\pm$ 0.04
Fathom 64, session-calibrated	74	0.54 $\pm$ 0.05
Fathom 80	92	0.60 $\pm$ 0.05
Fathom 80, agent-calibrated	92	0.67 $\pm$ 0.04
Fathom 80, session-calibrated	92	0.64 $\pm$ 0.04
Double Sparsity $c=32$	136	0.55 $\pm$ 0.04
Double Sparsity $c=32$, agent-calibrated	136	0.53 $\pm$ 0.04
SparQ $r=32$	136	0.60 $\pm$ 0.04